ServiceNow Client-Side Scripting

Intensive 5-Hour Mastery Study Plan

This intensive syllabus is designed to walk you step-by-step through all critical ServiceNow client-side APIs, execution boundaries, and debugging techniques. Complete the practical exercise for each hour to cement your skills.

Syllabus Plan Overview

Hour	Subject Area Covered
Hour 1	Form Control (g_form) & Banner Notifications
Hour 2	User Profiles (g_user) & Script Trigger Types (onLoad, onChange, onSubmit)
Hour 3	UI Policies vs. Client Scripts (Execution precedence & configurations)
Hour 4	Client-Server Bridge (Asynchronous GlideAjax calls & g_scratchpad variables)
Hour 5	Browser Redirections, Overlay Modals, & JS Executor debugging (Ctrl + Shift + J)

Hour 1: Form Control (g_form) & Notifications

g_form (GlideForm) is the primary class used to manipulate form fields and display messages to users on live records.

- Field Properties:
  - g_form.setDisplay('field', false): Hides field and collapses page space (recommended).
  - g_form.setVisible('field', false): Hides field but leaves page layout gaps.
  - g_form.setMandatory('field', true): Forces user input before saving.
  - g_form.setReadOnly('field', true): Locks the input field.

- Messaging APIs:
  - g_form.addInfoMessage() / addErrorMessage(): Banners at the top of the form.
  - g_form.showFieldMsg('field', 'msg', 'type'): Message directly below an input field.

Hour 1 Practical Task

Write an onChange client script on the Incident table. If Urgency is set to 1 (High): hide Configuration Item, make Description mandatory, and show an error banner.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') return;

    if (newValue === '1') { // 1 represents High Urgency
        g_form.setMandatory('description', true);
        g_form.setDisplay('cmdb_ci', false);
        g_form.addErrorMessage('Urgency is set to High! Describe the issue.');
        g_form.showFieldMsg('short_description', 'Ensure summary is specific', 'info');
    } else {
        g_form.setMandatory('description', false);
        g_form.setDisplay('cmdb_ci', true);
        g_form.clearMessages();
        g_form.hideFieldMsg('short_description');
    }
}
```

Hour 2: User Profiles (g_user) & Script Trigger Types

Learn the client-side user context and when Client Scripts run:

- Script Trigger Types:
  - onLoad: Runs when a form finishes rendering.
  - onChange: Runs when a monitored field changes value.
  - onSubmit: Runs when the user saves the form. Returning 'false' halts the save transaction.
  - onCellEdit: Runs when double-clicking record cells inside list grids.

- User Role Checks:
  - g_user.hasRole('role'): Checks role; automatically returns true for Administrators.
  - g_user.hasRoleExactly('role'): Checks explicit assignment; returns false for Admins if they don't hold the role explicitly.

Hour 2 Practical Task

Create an onSubmit script on Change Request. If the user is not an Administrator and Planned End Date is empty, block submission using return false.

```
function onSubmit() {
    var isAdmin = g_user.hasRoleExactly('admin');
    var plannedEnd = g_form.getValue('planned_end_date');

    if (!isAdmin && plannedEnd === '') {
        g_form.addErrorMessage('Only Admins can submit changes without Planned End Dates.');
        return false; // Aborts record save database submission!
    }
    return true; // Continues transaction
}
```

Hour 3: UI Policies vs. Client Scripts

- UI Policies:
  UI Policies are condition-based rules (e.g. State is Closed) that toggle field properties without writing code. Use them as the first choice for simple form controls because they are lightweight and performant.

- Client Scripts:
  Used when logic requires complex checks, validations, or server requests.

- Execution Precedence:
  1. onLoad Client Scripts execute first.
  2. UI Policies execute second (overriding onLoad controls).
  3. onChange Client Scripts execute third as fields update.

NOTE: If a Client Script and UI Policy modify the same field, the UI Policy will take precedence on form load. Avoid setting conflicting rules in both script types to prevent visual issues.

Hour 3 Practical Task

1. Configure a UI Policy: Condition is 'State IS On Hold'. Actions: Make 'Hold Reason' mandatory and visible.
2. Create an onChange Client Script on 'State' that attempts to make 'Hold Reason' optional.
3. Save both and open the form. Confirm the UI Policy overrides the Client Script.

Hour 4: The Client-Server Bridge (GlideAjax & g_scratchpad)

- GlideAjax:
  Used to query server tables without freezing the user's browser. It calls a client-callable Script Include and returns values asynchronously to a callback function.

- g_scratchpad:
  A data object loaded by a server-side Display Business Rule during form load. This pre-loads server data and passes it to the client, eliminating unnecessary GlideAjax database hits.

Hour 4 Practical Task

Create a client script that gets the Caller's Manager using GlideAjax.

```
// Server-Side Script Include (Mark 'Client Callable' as True)
var GetUserInfo = Class.create();
GetUserInfo.prototype = Object.extendsObject(AbstractAjaxProcessor, {
    getManager: function() {
        var userId = this.getParameter('sysparm_user_id');
        var userGR = new GlideRecord('sys_user');
        if (userGR.get(userId)) {
            return userGR.getDisplayValue('manager');
        }
        return '';
    },
    type: 'GetUserInfo'
});
```

```
// Client-Side Client Script (onChange of caller_id)
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') return;

    var ga = new GlideAjax('GetUserInfo');
    ga.addParam('sysparm_name', 'getManager');
    ga.addParam('sysparm_user_id', newValue);
    ga.getXMLAnswer(function(answer) {
        g_form.setValue('u_manager_name', answer);
    });
}
```

Hour 5: Navigation, Modals, & Debugging

- g_navigation:
Used to open frames or redirect URLs client-side (e.g. g_navigation.open(url, target)).

- GlideModal:
Provides clean modal popup boxes inside ServiceNow. Use this instead of raw javascript alert() popups to show confirmations or prompts.

- JavaScript Executor:
Press Ctrl + Shift + J on any ServiceNow form to open the Executor. You can write and run client-side JavaScript codes directly on the page.

Hour 5 Practical Task

Open the Javascript Executor (Ctrl + Shift + J) on any form and run this code to show a modal confirmation dialogue box:

```
var modal = new GlideModal('glide_confirm_basic');
modal.setTitle('Escalation Confirm');
modal.setPreference('title', 'Confirm Escalation?');
modal.setPreference('onPromptComplete', function() {
    g_form.addInfoMessage('Escalation confirmed!');
});
modal.render();
```